

Single-VM Architecture | 6-Agent Claude Code Swarm | World-Class Wiring

Digital Principles Corp | March 28, 2026

CLASSIFICATION: INTERNAL | VERSION 1.0

This document contains the complete implementation specification for deploying the Menagerie platform on a single Azure VM with 24/7 uptime, world-class HTTP performance, and a 6-agent Claude Code deployment swarm. Designed to be dropped directly into Claude Code as an execution plan.

TABLE OF CONTENTS

#	Section	Coverage
01	Executive Summary	Architecture decisions, cost, timeline
02	Architecture Overview	VM stack, service topology, data flow
03	Wiring Diagrams	Request flow, upload pipeline, AI pipeline
04	World-Class Speed Layer	HTTP/2, connection pooling, compression
05	Error Prevention Matrix	Pre-launch error catalog with mitigations
06	6-Agent Claude Code Swarm	Agent roles, rules, prompts, coordination
07	Full Setup Script	One-run provisioning script for the VM
08	Claude Code Drop-In Rules	CLAUDE.md rules file for execution
09	Cost Analysis	Monthly burn, reserved pricing, optimization
10	Implementation Timeline	30-min to 2-hour execution blocks

01 EXECUTIVE SUMMARY

The Menagerie platform deploys on a single Azure D4s_v5 VM (4 vCPU, 16GB RAM) running 24/7. All services run on one machine: Nginx reverse proxy, FastAPI application server, PostgreSQL 16 (51 tables), and Redis cache. External Azure services (Blob Storage, AI Search, Azure OpenAI) are called via SDK from the VM. The ANVIL fleet (three M1 MacBook Pros) is fully replaced by this VM.

Key Decisions

Decision	Choice	Rationale
Compute	Single VM (D4s_v5)	Always hot, no cold starts, sub-5ms local DB
API Framework	FastAPI + uvicorn	Async, streaming SSE, 4 workers, auto-restart
Database	PostgreSQL 16 (local)	Zero-latency queries, pgvector for embeddings
Cache	Redis (local)	Sub-1ms reads, session/query cache
File Storage	Azure Blob (external)	Durable, SAS token direct upload
Search	Azure AI Search (external)	Managed indexing, semantic search
AI	Azure OpenAI (external)	GPT-4o/Claude streaming via SSE
Reverse Proxy	Nginx + HTTP/2	TLS termination, compression, multiplexing
Frontend	Tauri (Rust + Web)	Native desktop, single HTTP client
Deploy	SSH + git pull + systemctl	10-second deploys, no CI/CD overhead

Cost Summary

Service	Cost	Notes
Azure VM D4s_v5 (reserved 1yr)	\$85/mo	4 vCPU, 16GB, always on
Azure Blob Storage	\$10/mo	100GB estimated
Azure AI Search (Basic)	\$75/mo	1 replica, 1 partition
Azure OpenAI	~\$50/mo	Usage-based, streaming
Domain + TLS (Certbot)	\$0/mo	Let's Encrypt free
TOTAL	\$220/mo	Reserved pricing, all-in

02 ARCHITECTURE OVERVIEW

Service Topology

```
=====
MENAGERIE VM STACK
=====

TAURI DESKTOP APP (Rust + WebView)
|
| HTTPS (HTTP/2, TLS 1.3, keep-alive)
|
[NGINX] ---- reverse proxy, gzip/brotli, rate limit
|
[FASTAPI] -- uvicorn, 4 workers, systemd managed
|
+-----+-----+-----+-----+
| | | | |
[PG 16] [REDIS] [BLOB] [AI [AZURE
local local SDK SEARCH] OPENAI]
51 tbl cache SAS SDK streaming
pgvec <1ms token 50-200ms SSE
<5ms upload

=====

BACKGROUND WORKERS (asyncio / Celery)
- Vectorization (Azure OpenAI embeddings)
- Index updates (AI Search push)
- Backup cron (pg_dump -> Blob nightly)

=====
```

Endpoint Map

Endpoint	Purpose	Zone	Target Latency
POST /api/auth/token	Issue JWT	GREEN	<10ms
GET /api/health	Health check	GREEN	<1ms
POST /api/crud/{table}	Create record	YELLOW	<10ms
GET /api/crud/{table}/{id}	Read record	GREEN	<5ms
PUT /api/crud/{table}/{id}	Update record	YELLOW	<10ms
DELETE /api/crud/{table}/{id}	Delete record	RED	<10ms
POST /api/upload/sas	Generate SAS token	GREEN	<5ms
POST /api/upload/confirm	Confirm + trigger vectorize	YELLOW	<10ms

Endpoint	Purpose	Zone	Target Latency
POST /api/search	AI Search query	GREEN	50-200ms
POST /api/chat	AI chat (SSE stream)	GREEN	300ms TTFT
GET /api/vectors/{doc_id}	Get embeddings	GREEN	<20ms
POST /api/batch/vectorize	Batch vectorize (async)	YELLOW	Async
GET /api/jobs/{job_id}	Check async job status	GREEN	<5ms

03 WIRING DIAGRAMS

Diagram A: Standard CRUD Request Flow

```
TAURI VM
| |
|-- POST /api/crud ---->|
| {table: 'pets', |
| data: {...}} |
| |
| [NGINX]
| | HTTP/2 demux
| | gzip decompress
| |
| [FASTAPI]
| | validate input
| | check Redis cache
| | HIT? -> return cached
| | MISS? -> query Postgres
| | write to Postgres (<5ms)
| | invalidate Redis key
| | gzip response
| |
|<-- 200 OK -----|
| {id: 42, ...} |
| |
TOTAL: 10-20ms end-to-end
```

Diagram B: File Upload Pipeline (SAS Direct)

```
TAURI VM AZURE BLOB
| | |
|-- POST /upload/sas -->| |
| {filename, size} | |
| | |
| [generate SAS token, 2ms] |
| | |
|<-- {sas_url, token} --| |
| | |
|-- PUT (file bytes) ---|----->|
| direct-to-blob | (bypasses VM) |
| | |
|<-- 201 Created -----|-----|
| | |
|-- POST /upload/confirm| |
| {blob_url} | |
| [save metadata to PG] |
```

```

| [trigger async vectorize] |
| | |
|<-- 202 Accepted -----| |
| {job_id} | |
| | |
USER SEES: instant upload (file never touches VM)

```

Diagram C: AI Chat Streaming (SSE)

```

TAURI VM AZURE OPENAI
| | |
|-- POST /api/chat ---->| |
| {prompt, context} | |
| | |
| [build messages array] |
| [fetch context from PG/Redis] |
| [call Azure OpenAI stream] |
| |-- stream request ----->|
| | |
|<-- SSE: token 1 -----|<-- chunk 1 -----|
|<-- SSE: token 2 -----|<-- chunk 2 -----|
|<-- SSE: token 3 -----|<-- chunk 3 -----|
| ... | ... |
|<-- SSE: [DONE] -----|<-- finish_reason:stop --|
| | |
USER SEES: first token in ~300ms, text streams live

```

Diagram D: Background Vectorization Pipeline

```
[UPLOAD CONFIRM triggers]
|
v
[ASYNC WORKER]
|
+-- 1. Fetch blob from Azure Blob Storage
|
+-- 2. Extract text (pdfplumber / python-docx / etc)
|
+-- 3. Chunk text (512 token windows, 50 overlap)
|
+-- 4. Call Azure OpenAI Embeddings API
| model: text-embedding-3-large
| dimensions: 3072
|
+-- 5. Store vectors in PostgreSQL (pgvector)
| INSERT INTO embeddings (doc_id, chunk, vector)
|
+-- 6. Push to Azure AI Search index
| POST /indexes/docs/docs/index
|
+-- 7. Update job status -> 'completed'
|
+-- 8. Notify via WebSocket/SSE (optional)

TOTAL: 5-30 seconds per document (user never waits)
```

04 WORLD-CLASS SPEED LAYER

Three configuration-level changes that drop subsequent request latency from 80-120ms to 5-15ms. No code changes. No architecture changes. Pure config.

Optimization 1: HTTP/2 Multiplexing

HTTP/1.1 opens a new TCP+TLS connection per request (50-150ms handshake overhead). HTTP/2 opens ONE persistent connection and multiplexes all requests through it. First request pays the handshake. Every subsequent request: near-zero overhead.

```
listen 443 ssl http2;
ssl_certificate /etc/letsencrypt/live/DOMAIN/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/DOMAIN/privkey.pem;
ssl_protocols TLSv1.3;
ssl_prefer_server_ciphers off;
```

Optimization 2: Connection Pooling (Tauri Side)

Never create a new HTTP client per request. Hold a single request::Client for the entire application lifetime. It pools connections automatically.

```
// In Tauri Rust backend:
lazy_static! {
    static ref CLIENT: request::Client = request::Client::builder()
        .http2_prior_knowledge()
        .pool_max_idle_per_host(10)
        .tcp_keepalive(Duration::from_secs(60))
        .build().unwrap();
}
```

Optimization 3: Response Compression

Enable gzip/brotli on Nginx. JSON responses compress 70-80%. A 50KB response becomes 10KB on the wire. Tauri's request decompresses automatically.

```
gzip on;
gzip_vary on;
gzip_proxied any;
gzip_comp_level 6;
gzip_types application/json text/plain text/css application/javascript;
gzip_min_length 256;
```

Latency Impact

Operation	Before	After	Why
CRUD read (cached)	80-120ms	<1ms	Redis localhost
CRUD read (DB)	80-120ms	<5ms	Postgres localhost
CRUD write	100-150ms	<10ms	Postgres localhost

Operation	Before	After	Why
File upload (10MB)	5-10s	<2s	SAS direct-to-blob
Search query	300-500ms	50-200ms	AI Search, no middleware hop
AI first token	1-3s	~300ms	SSE streaming
Subsequent HTTP call	80-120ms	5-15ms	HTTP/2 + keep-alive

05 ERROR PREVENTION MATRIX

Every known failure mode cataloged with prevention strategy. No errors during dev. Fix them before they happen.

ID	Failure Mode	Risk	Prevention + Mitigation
E-001	VM cold boot after restart	HIGH	systemd services with Restart=always + WantedBy=multi-user.target. All services auto-start on boot. Test: sudo reboot, verify all services up.
E-002	Postgres connection exhaustion	HIGH	PgBouncer or max_connections=200 in postgresql.conf. FastAPI uses connection pool (asyncpg pool, min=5, max=20). Never open raw connections.
E-003	Disk fills up	HIGH	Never store uploads on VM. All files to Blob. Log rotation: logrotate.d config, 7-day retention, compress. Monitoring: Azure disk alert at 80%.
E-004	SSL cert expires	MEDIUM	Certbot auto-renewal cron: 0 0 1 * * certbot renew --nginx. Test: certbot renew --dry-run after setup.
E-005	FastAPI crashes	MEDIUM	systemd Restart=always, RestartSec=3. Uvicorn --workers 4 (one dies, three serve). Health check endpoint + UptimeRobot alert.
E-006	Redis OOM	LOW	maxmemory 2gb + maxmemory-policy allkeys-lru in redis.conf. Redis evicts least-used keys automatically.
E-007	CORS errors from Tauri	HIGH	Nginx add_header Access-Control-Allow-Origin for Tauri origin. Include in setup script. Test with curl -H 'Origin: tauri://localhost'.
E-008	Large file upload timeout	MEDIUM	SAS token pattern bypasses VM entirely. Tauri uploads direct to Blob. Nginx client_max_body_size only matters for non-file endpoints (set 10m).

ID	Failure Mode	Risk	Prevention + Mitigation
E-009	Azure OpenAI rate limit (429)	MEDIUM	Exponential backoff in SDK call (tenacity library). Queue requests in Redis. Fallback: route to Anthropic API if Azure throttles.
E-010	SSH brute force	HIGH	NSG: allow SSH only from your IP. fail2ban installed. SSH key auth only, password auth disabled.
E-011	Postgres data loss	CRITICAL	Nightly pg_dump cron to Blob storage. Azure VM disk snapshots weekly. Test restore monthly.
E-012	Background worker silent failure	MEDIUM	Job status table in Postgres. Workers update status on start/complete/fail. Stale job detector: any job 'running' > 10min = alert.
E-013	DNS propagation delay	LOW	Set low TTL (300s) on A record before migration. During transition, allow both IP and domain access.

ID	Failure Mode	Risk	Prevention + Mitigation
E-01 4	Python dependency conflict	LOW	Use venv for FastAPI. Pin all versions in requirements.txt. Never pip install globally.
E-01 5	Azure SDK auth token expiry	MEDIUM	Use DefaultAzureCredential with managed identity on VM. Tokens auto-refresh. Never hardcode keys in code.

06 6-AGENT CLAUDE CODE SWARM

Six specialized Claude Code agents execute in parallel. Each agent has a single responsibility, its own workspace rules, and a completion target. No agent touches another agent's files. Coordination via a shared STATUS.md file.

Agent Roster

Agent	Role	Scope	Zone	Time	Tools
AGENT 1	INFRA	VM provisioning, Nginx, systemd, firewall, TLS	GREEN	30 min	az CLI, SSH, systemd, certbot, ufw
AGENT 2	DATABASE	PostgreSQL 16 install, 51-table schema, pgvector, PgBouncer, backup cron	YELLOW	30 min	psql, pg_dump, pgvector extension
AGENT 3	API	FastAPI app: all 13 endpoints, Redis integration, error handling, streaming SSE	YELLOW	1 hour	Python, FastAPI, uvicorn, redis-py, asyncpg
AGENT 4	STORAGE	Azure Blob wiring, SAS token generation, upload/download helpers, AI Search indexer	GREEN	30 min	azure-storage-blob SDK, azure-search SDK
AGENT 5	AI	Azure OpenAI streaming, embeddings pipeline, vectorization worker, RAG chain	YELLOW	30 min	openai SDK, tiktoken, asyncio workers
AGENT 6	SECURITY	NSG rules, fail2ban, SSH hardening, managed identity, secrets management, monitoring	RED	30 min	Azure CLI, ufw, fail2ban, Azure Monitor

Agent Coordination Protocol

```
=====
AGENT SWARM COORDINATION
=====

AGENT 1 (INFRA) ----+
| | Phase 1: VM + OS + Nginx
v | (must complete before others)
[VM READY] |
| |
+-----+-----+-----+
| | | | |
AG-2 AG-3 AG-4 AG-5 AG-6
```

```

(DB) (API) (STOR) (AI) (SEC)
| | | | |
v v v v v
[Phase 2: All agents run in parallel]
| | | | |
+-----+-----+-----+-----+
|
v
[INTEGRATION TEST]
|
v
[DEPLOY TO PRODUCTION]
=====

```

Agent Rules (Shared CLAUDE.md)

```

# MENAGERIE VM DEPLOYMENT - AGENT RULES

## Universal Rules (All Agents)
- Zone system: RED = human approval, YELLOW = test required, GREEN = autonomy
- Time blocks: 1/5/10/30/60 min. Over 2 hours = WARNING
- No hallucinations. Every command must be tested before marking complete
- Update STATUS.md after each completed step
- If blocked, write BLOCKED:{reason} to STATUS.md and stop
- Never modify files owned by another agent
- All secrets via environment variables, never hardcoded
- Log every action to /var/log/menagerie/agent-{N}.log

```

Agent 1: INFRA — Prompt

You are AGENT-1-INFRA. Your job: provision the Azure VM and configure the OS layer.

TASKS:

1. Create resource group 'menagerie-rg' in eastus
2. Provision D4s_v5 VM with Ubuntu 24.04
3. Install Nginx, configure HTTP/2 + TLS 1.3 via Certbot
4. Enable gzip (level 6, json/text/css/js, min 256 bytes)
5. Configure systemd service template for FastAPI
6. Set up logrotate (7-day retention, compress)
7. Configure UFW: allow 443/tcp, allow 22/tcp from ADMIN_IP only
8. Write nginx.conf to /etc/nginx/sites-available/menagerie
9. Verify: curl -I https://DOMAIN returns 200 with h2
10. Update STATUS.md: INFRA=COMPLETE

DEPENDENCIES: None. You run first.

TIME TARGET: 30 minutes

ZONE: GREEN (no production data yet)

Agent 2: DATABASE — Prompt

You are AGENT-2-DATABASE. Your job: install and configure PostgreSQL 16 with full schema.

WAIT FOR: STATUS.md shows INFRA=COMPLETE

TASKS:

1. Install PostgreSQL 16 + pgvector extension
2. Create database 'menagerie' and user 'menagerie_app'
3. Load 51-table schema from schema.sql
4. Configure postgresql.conf: max_connections=200, shared_buffers=4GB, work_mem=64MB, effective_cache_size=12GB
5. Enable PgBouncer (pool_mode=transaction, max_client_conn=400)
6. Create backup script: pg_dump -> gzip -> az storage blob upload
7. Add cron: 0 3 * * * /opt/menagerie/backup.sh
8. Install Redis 7, configure maxmemory=2gb, allkeys-lru
9. Create systemd services for PostgreSQL and Redis
10. Verify: psql -c 'SELECT count(*) FROM information_schema.tables'
11. Update STATUS.md: DATABASE=COMPLETE

ZONE: YELLOW (schema changes require review)

Agent 3: API — Prompt

You are AGENT-3-API. Your job: build the complete FastAPI application.

WAIT FOR: STATUS.md shows INFRA=COMPLETE

TASKS:

1. Create Python 3.12 venv at /opt/menagerie/venv
2. Install: fastapi uvicorn asyncpg redis aiohttp python-multipart pydantic
3. Build FastAPI app with 13 endpoints (see endpoint map)
4. Implement connection pooling: asyncpg pool (min=5, max=20)
5. Implement Redis caching layer with 5-min TTL for reads
6. Implement SSE streaming endpoint for /api/chat
7. Add request validation with Pydantic models
8. Add structured error responses (JSON, status codes, error codes)
9. Add request logging middleware (method, path, status, latency_ms)
10. Create systemd service: menagerie-api.service
11. Verify: all 13 endpoints return expected responses
12. Update STATUS.md: API=COMPLETE

ZONE: YELLOW (core application logic)

Agent 4: STORAGE — Prompt

You are AGENT-4-STORAGE. Your job: wire Azure Blob Storage and AI Search.

WAIT FOR: STATUS.md shows INFRA=COMPLETE

TASKS:

1. Install azure-storage-blob, azure-search-documents SDKs
2. Create Blob container 'menagerie-uploads' with private access
3. Build SAS token generator (5-min expiry, write-only, per-blob)
4. Build upload confirmation handler (save metadata to PG)
5. Create AI Search index 'menagerie-docs' with schema
6. Build index push function (document -> AI Search)
7. Build search query function with filters and facets
8. Wire storage helpers into FastAPI app (import as module)
9. Verify: generate SAS token, upload test file, confirm in Blob
10. Verify: push test doc to AI Search, query returns result
11. Update STATUS.md: STORAGE=COMPLETE

ZONE: GREEN (no production data risk)

Agent 5: AI — Prompt

You are AGENT-5-AI. Your job: wire Azure OpenAI and build the vectorization pipeline.

WAIT FOR: STATUS.md shows INFRA=COMPLETE

TASKS:

1. Install openai, tiktoken SDKs
2. Build streaming chat endpoint using Azure OpenAI (SSE)
3. Build embeddings function (text-embedding-3-large, 3072 dims)
4. Build text chunker (512 tokens, 50 overlap, tiktoken counter)
5. Build vectorization worker (async): fetch blob -> extract -> chunk -> embed -> store
6. Store vectors in PostgreSQL pgvector (HNSW index, cosine distance)
7. Build RAG chain: query -> vector search -> context assembly -> LLM call
8. Add retry logic with tenacity (3 retries, exponential backoff)
9. Add fallback: if Azure OpenAI 429, route to Anthropic API
10. Verify: embed test document, retrieve via similarity search
11. Verify: chat endpoint streams tokens correctly
12. Update STATUS.md: AI=COMPLETE

ZONE: YELLOW (AI pipeline is core functionality)

Agent 6: SECURITY — Prompt

You are AGENT-6-SECURITY. Your job: harden the VM and set up monitoring.

WAIT FOR: STATUS.md shows INFRA=COMPLETE

TASKS:

1. Configure NSG: 443 open, 22 from ADMIN_IP only, deny all else
2. Install and configure fail2ban (SSH: 3 attempts, 1hr ban)
3. Disable password SSH auth, key-only
4. Set up Azure Managed Identity for VM
5. Configure DefaultAzureCredential for all SDK calls
6. Create /opt/menagerie/.env with all secrets, chmod 600
7. Set up Azure Monitor agent + disk/CPU/memory alerts
8. Configure UptimeRobot (free) for /api/health endpoint
9. Set up unattended-upgrades for security patches
10. Run Lynis security audit, fix any HIGH findings
11. Verify: nmap scan from external shows only 443 open
12. Update STATUS.md: SECURITY=COMPLETE

ZONE: RED (security changes require human review)

Single script. SSH in once. Run it. Walk away for 20 minutes. Come back to a fully running production server.

```
#!/bin/bash
set -eou pipefail
# ████████████████████████████████████████████████████████████████
# MENAGERIE VM SETUP - ONE SCRIPT, ONE RUN
# ████████████████████████████████████████████████████████████████

DOMAIN='menagerie.yourdomain.com'
DB_NAME='menagerie'
DB_USER='menagerie_app'
DB_PASS='CHANGE_ME_STRONG_PASSWORD'
ADMIN_EMAIL='alan@digitalprinciples.com'

echo '=== [1/8] System Update ==='
sudo apt update && sudo apt upgrade -y
sudo apt install -y nginx postgresql-16 postgresql-16-pgvector \
    redis-server python3.12 python3.12-venv python3-pip \
    certbot python3-certbot-nginx fail2ban ufw logrotate curl

echo '=== [2/8] PostgreSQL ==='
sudo -u postgres psql -c "CREATE DATABASE $DB_NAME;"
sudo -u postgres psql -c "CREATE USER $DB_USER WITH PASSWORD '$DB_PASS';"
sudo -u postgres psql -c "GRANT ALL ON DATABASE $DB_NAME TO $DB_USER;"
sudo -u postgres psql -d $DB_NAME -c 'CREATE EXTENSION IF NOT EXISTS vector;'

echo '=== [3/8] Redis ==='
sudo sed -i 's/# maxmemory */maxmemory 2gb/' /etc/redis/redis.conf
sudo sed -i 's/# maxmemory-policy */maxmemory-policy allkeys-lru/' /etc/redis/redis.conf
sudo systemctl restart redis

echo '=== [4/8] Python + FastAPI ==='
sudo mkdir -p /opt/menagerie
python3.12 -m venv /opt/menagerie/venv
source /opt/menagerie/venv/bin/activate
pip install fastapi uvicorn[standard] asynctpg redis aiohttp \
    python-multipart pydantic azure-storage-blob \
    azure-search-documents openai tiktoken tenacity

echo '=== [5/8] Nginx + TLS ==='
# (nginx config written separately, see Agent 1)
sudo certbot --nginx -d $DOMAIN --non-interactive --agree-tos -m $ADMIN_EMAIL

echo '=== [6/8] Firewall ==='
sudo ufw default deny incoming
sudo ufw allow 443/tcp
sudo ufw allow 22/tcp # restrict to IP in NSG
sudo ufw --force enable

echo '=== [7/8] fail2ban ==='
sudo systemctl enable fail2ban
sudo systemctl start fail2ban

echo '=== [8/8] Systemd Services ==='
# FastAPI service
cat <<EOF | sudo tee /etc/systemd/system/menagerie-api.service
[Unit]
Description=Menagerie FastAPI
After=network.target postgresql.service redis.service
[Service]
Type=exec
User=alan
WorkingDirectory=/opt/menagerie
EnvironmentFile=/opt/menagerie/.env
```

```
ExecStart=/opt/menagerie/venv/bin/uvicorn app.main:app \
  --host 127.0.0.1 --port 8000 --workers 4
Restart=always
RestartSec=3
[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable menagerie-api
sudo systemctl start menagerie-api

echo '=== SETUP COMPLETE ==='
echo 'All services running. Verify: curl https://$DOMAIN/api/health'
```

08 CLAUDE CODE DROP-IN RULES

Copy this into your CLAUDE.md. It gives Claude Code full context to execute any task on the Menagerie VM stack.

```
# MENAGERIE VM - CLAUDE CODE RULES

## Architecture
- Single Azure VM: D4s_v5, Ubuntu 24.04, East US
- Stack: Nginx (HTTP/2) -> FastAPI (uvicorn, 4 workers) -> PostgreSQL 16 + Redis
- External: Azure Blob (SAS upload), AI Search, Azure OpenAI (streaming)
- Deploy: SSH -> git pull -> systemctl restart menagerie-api
- All code lives at /opt/menagerie/

## File Ownership
- /etc/nginx/sites-available/menagerie -> AGENT-1 (INFRA)
- /opt/menagerie/schema.sql -> AGENT-2 (DATABASE)
- /opt/menagerie/app/ -> AGENT-3 (API)
- /opt/menagerie/app/storage/ -> AGENT-4 (STORAGE)
- /opt/menagerie/app/ai/ -> AGENT-5 (AI)
- /opt/menagerie/.env, firewall, fail2ban -> AGENT-6 (SECURITY)

## Zone Rules
- RED: .env, firewall rules, SSL certs, backup scripts -> HUMAN APPROVAL
- YELLOW: schema changes, API endpoints, AI pipeline -> REQUIRE TESTS
- GREEN: docs, utilities, new features, monitoring -> FULL AUTONOMY

## Speed Rules
- HTTP/2 enabled on Nginx (listen 443 ssl http2)
- gzip on all JSON responses (level 6, min 256 bytes)
- Tauri client uses single request::Client with connection pooling
- Redis caches all reads with 5-min TTL
- PgBouncer for connection pooling (transaction mode)
- SAS token direct-to-blob for file uploads
- SSE streaming for all AI responses

## Error Prevention
- All services: systemd Restart=always, RestartSec=3
- Postgres: max_connections=200, backup cron nightly
- Redis: maxmemory=2gb, allkeys-lru eviction
- Nginx: client_max_body_size=10m (files bypass via SAS)
- SSL: certbot auto-renew cron monthly
- Monitoring: Azure Monitor + UptimeRobot on /api/health

## Time Blocks
- 1 min: config tweak, single query
- 5 min: add endpoint, fix bug
- 10 min: new feature, refactor module
- 30 min: full pipeline, integration
- 1 hour: major feature, migration
- >2 hours: WARNING - 'Could fuck your day Alan'

## Merge Rules (A/B/C/D/E)
- A: Deployment (faster/safer shipping)
- B: Revenue (money in)
- C: Cost (money out reduction)
- D: Organization (clarity, maintainability)
- E: Legal (risk reduction, compliance)
```

09 COST ANALYSIS

Service	Tier	Cost	Notes
Azure VM D4s_v5	Pay-as-you-go	\$140/mo	Always on, 4 vCPU, 16GB
Azure VM D4s_v5	1-year reserved	\$85/mo	39% savings
Azure VM D4s_v5	3-year reserved	\$55/mo	61% savings
Managed Disk (128GB P10)	Standard	\$20/mo	OS + app + DB data
Azure Blob Storage	Hot tier	\$10/mo	~100GB uploads
Azure AI Search	Basic	\$75/mo	1 replica, 15GB index
Azure OpenAI	Usage	~\$50/mo	GPT-4o chat + embeddings
Domain + DNS	Azure DNS	\$1/mo	Zone hosting
TLS Certificate	Let's Encrypt	\$0/mo	Auto-renew via Certbot
Monitoring	Azure Monitor Basic	\$0/mo	Included with VM
UptimeRobot	Free tier	\$0/mo	50 monitors, 5-min checks

Pricing Model	Monthly Total	Annual Total
Pay-as-you-go	\$296/mo	\$3,552/yr
1-year reserved	\$241/mo	\$2,892/yr
3-year reserved	\$211/mo	\$2,532/yr

10 IMPLEMENTATION TIMELINE

Phase 1: Foundation (30 min)

Time	Agent	Task	Completion %
0:00-0:05	AGENT 1	az vm create + SSH key setup	95%
0:05-0:15	AGENT 1	apt install all packages	98%
0:15-0:25	AGENT 1	Nginx config + Certbot TLS	90%
0:25-0:30	AGENT 1	Verify HTTPS + HTTP/2 working	95%

Phase 2: Parallel Build (1 hour)

Time	Agent	Task	Completion %
0:30-0:45	AGENT 2	PostgreSQL + pgvector + schema load	90%
0:30-0:45	AGENT 6	Firewall + fail2ban + SSH hardening	95%
0:30-1:00	AGENT 3	FastAPI app (13 endpoints)	85%
0:45-1:00	AGENT 2	Redis + PgBouncer + backup cron	90%
0:45-1:15	AGENT 4	Blob wiring + SAS + AI Search index	85%
0:45-1:15	AGENT 5	OpenAI streaming + embeddings + RAG	80%
1:00-1:15	AGENT 6	Managed identity + monitoring + alerts	85%
1:15-1:30	AGENT 3	Integration with AG-4 and AG-5 modules	80%

Phase 3: Integration + Verification (30 min)

Time	Agent	Task	Completion %
1:30-1:45	ALL	End-to-end test: upload -> vectorize -> search -> chat	85%
1:45-1:55	AGENT 1	systemd enable all, reboot test, verify auto-start	95%
1:55-2:00	AGENT 6	Final security audit (Lynis), nmap verification	90%

TOTAL WALL CLOCK: 2 HOURS. Overall completion probability: 85%. Primary risk: Azure networking config (VNet, NSG rules, managed identity). Mitigation: Agent 6 handles all networking in parallel while other agents build.

POST-DEPLOY VERIFICATION CHECKLIST: curl /api/health returns 200. Upload test file via SAS, confirm in Blob. CRUD create/read/update/delete on test table. Chat endpoint streams tokens via SSE. Reboot VM, all services auto-restart. nmap shows only port 443 open. pg_dump backup completes and uploads to Blob.